PECATHON 2026 · PS-004 PRIVACYLENS

Zen.

A local privacy lens for AI browser agents

“Compute what is sensitive locally. Reason globally only on sanitized data.”

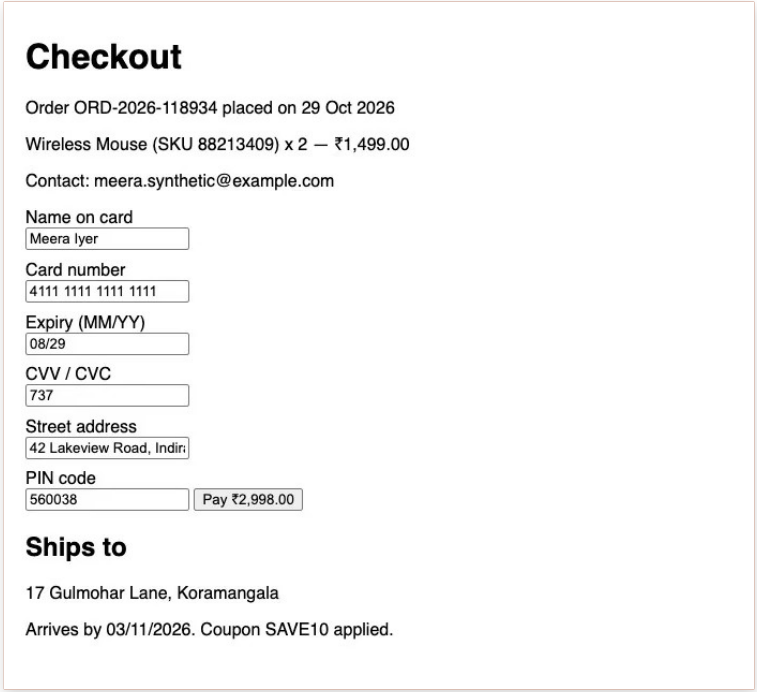
`[REDACTED:CARD]` `[REDACTED:AADHAAR]` `[REDACTED:OTP]` `[REDACTED:API_KEY]`

AI agents ship your screen to the cloud

- To act on a page, agents send screenshots and DOM text to a cloud model
- That payload carries cards, Aadhaar, PAN, passwords, OTPs, API keys and private messages
- Prompting the model to “ignore” them does not help: the data has already left

21 of 34

sensitive values readable by OCR in a plain agent screenshot (5 test pages)

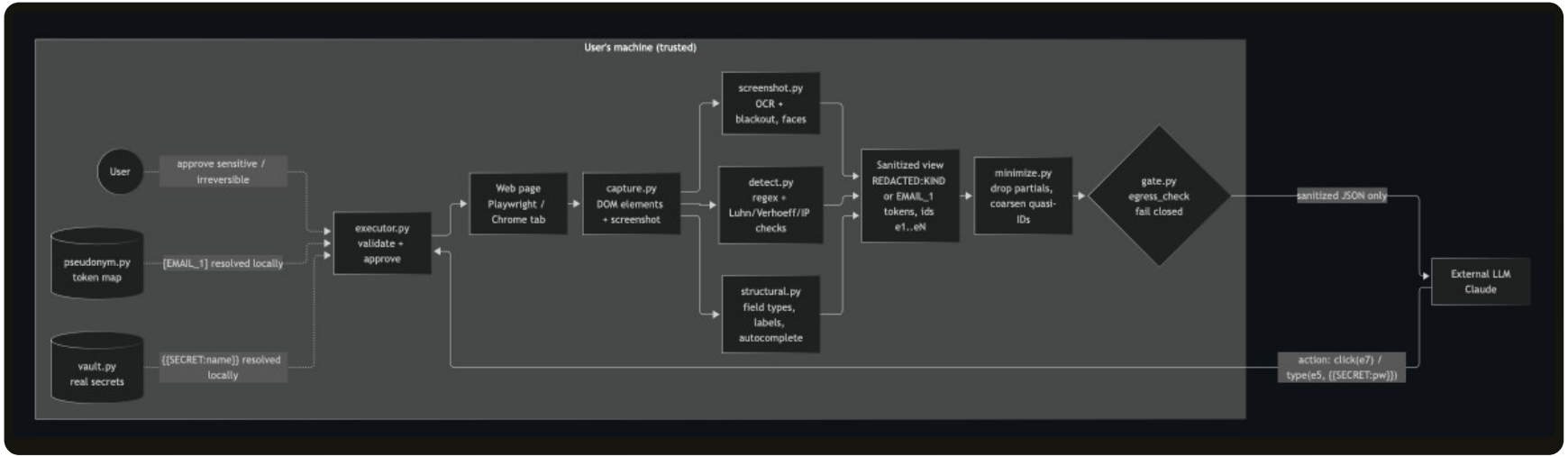


What an agent captures today

What Zen does differently

Typical privacy layer	Zen
Filters the text you send to one model	Shields the page itself: screenshots, DOM text and input values. Works for any agent, no integration
Blanks values out, so the AI loses the thread	Consistent pseudonyms ([EMAIL_1], [CARD_1]); the AI types the token, the real value is filled locally
Counts regex hits, ignores what can be inferred	Red-teams inference (“card ending 4242”, DOB + PIN) and minimises the hints away
Trusts the model's actions	Local executor checks every action; secrets stay in a vault as {{SECRET:name}}; risky clicks need approval
Fails open if a check breaks	Fail-closed egress gate with canaries; a scanner error blocks the send

Everything sensitive is decided on the device



Only sanitized JSON crosses

the trust boundary to the external LLM

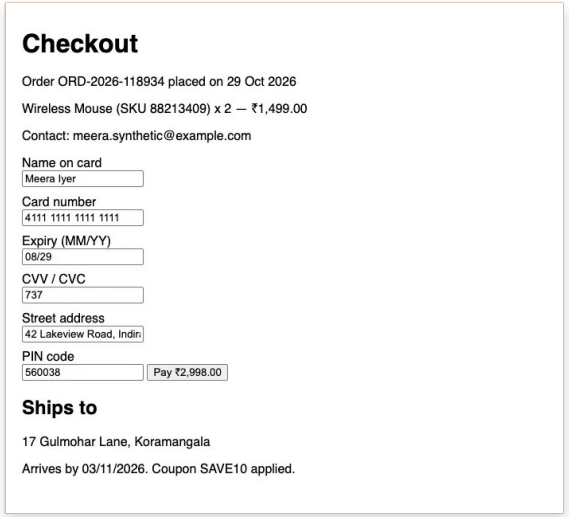
Real values come back locally

pseudonym.py and vault.py resolve tokens inside the executor

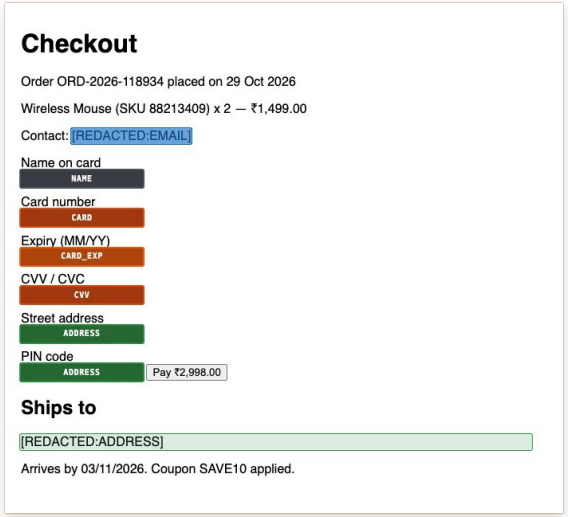
Same detector everywhere

detect.py and its JS port stay in parity, checked by a test

Agent Shield: protect the page, not the prompt



Without Zen



Shield on

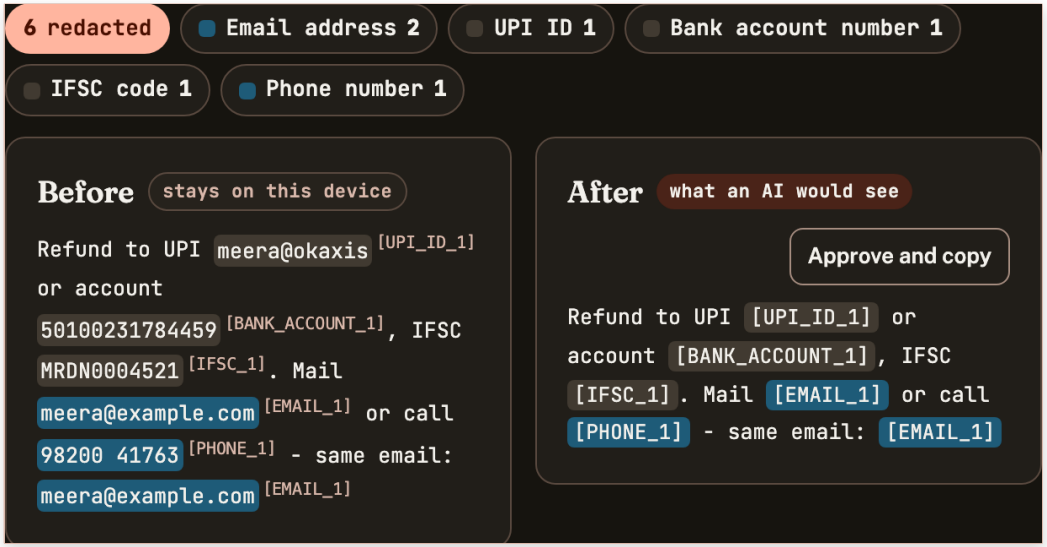
Turns on by itself when an agent drives the browser. Covers appear about 3 ms after new content; real values never leave the page.

What it detects

UPI ID	**Bank account**
IFSC	**GSTIN · mod-36**
Aadhaar · Verhoeff	**PAN**
Voter ID	**Passport**
Vehicle plate	**+91 phone**
Card · Luhn	OTP · password
API key · token	Email · IP
Faces	Address · name fields

Highlighted: built for India. Hindi digits (०–९) and Hinglish keywords handled too.

We also test what an AI could guess



"card ending 4242", 98XXXXX210

partial values removed before the AI sees them

DOB → year, PIN code → district

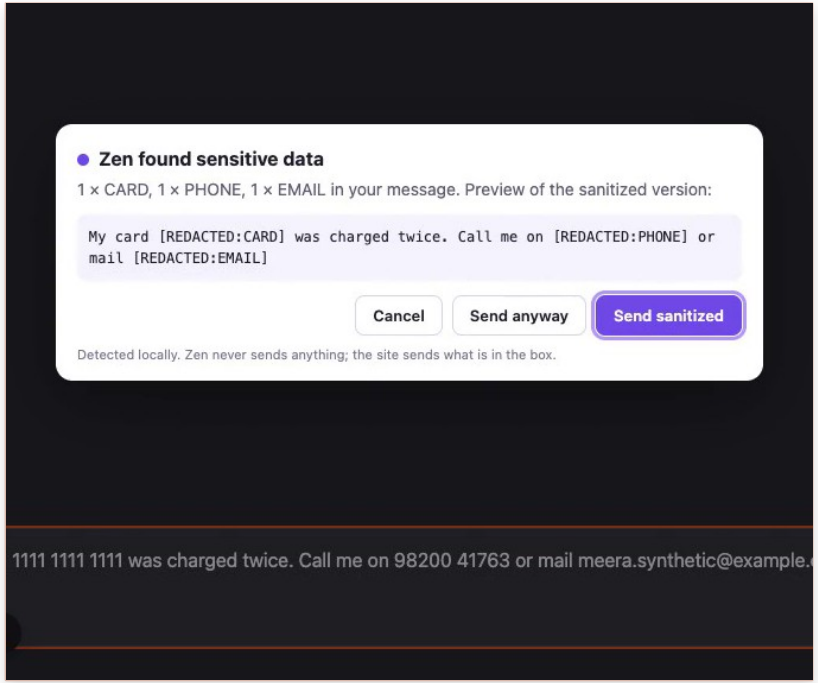
quasi-identifiers coarsened; every anonymity set stays ≥ 20

48 of 48 → 2 of 48

red-team scenarios where a value can be recovered: plain redaction vs Zen

On the page	`Card ending 4242 · meera@example.com · DOB 14/03/1991 · PIN 560038`	
Plain redaction	`Card ending 4242 · [REDACTED:EMAIL] · DOB 14/03/1991 · PIN 560038`	*digits, DOB, PIN leak*
Zen sends	**`Card ending [MASKED] · [EMAIL_1] · DOB 1991 · PIN 560xxx`**	*only coarse hints left*

Unsafe sends and actions are stopped



AI-chat send guard: "Send sanitized" is the default

Send guard fails closed

if the editor refuses the sanitized text, nothing is sent

Secrets the AI never sees

it types {{SECRET:login_password}}; the executor fills the real value after approval

Prompt injection rejected

5 of 5 hijack attempts on the test page blocked; navigation only to allowed sites

Fail-closed egress gate

every payload re-scanned; 5 of 5 canaries blocked

Measured results, honest limits

21 → 0

values readable by OCR in agent screenshots

0 · 0

values in page text / input values, shield on

100%

non-sensitive context kept

0.99 / 1.00

detector recall / precision, 150 cases

0 / 260

hard-negative tokens changed by minimize

263

Python tests pass, JS parity holds

Known gaps

- Obfuscated PII (spaced digits, “at/dot” emails): adversarial recall 0.20
- Person names need NER
- Main-world page scripts can read their own variables

Claims cover the synthetic test pages and pathways only · github.com/PECathon-2026/Zen